PROJECT MILESTONE & ROADMAP REPORT

Smart Resume Screening & Candidate Ranking System Using AI — Phase 1 & Roadmap

Project Phase: Phase 1 Completed	**Target Scope:** Weeks 1–4 Delivered	**Generated:** September 19, 2026	**Status:** On Schedule (100% Phase 1)

1. Executive Summary & Project Objectives

The **Smart Resume Screening and Candidate Ranking System Using AI** is an intelligent, automated recruitment platform engineered to streamline talent acquisition. The end-to-end architecture ingests multi-format resumes, performs layout-preserving text extraction, identifies named entities and technical competencies via NLP, computes semantic embeddings using Sentence-BERT, calculates cosine similarity ranking against job descriptions, and applies algorithmic fairness audits to eliminate hiring biases.

Current Status: Phase 1 (Weeks 1–4) has been **fully executed, tested, and verified**. The underlying microservice architecture, asynchronous persistence layer, hybrid OCR parsing pipeline, spaCy NER skills extractor, and responsive workstation UI are completely operational.

2. Detailed Work Completed (Phase 1)

• **Component 1: Project Skeleton & Configuration:** Configured clean repository root with .gitignore, multi-stage backend Dockerfile, docker-compose.yml for MongoDB and API orchestration, pydantic-settings environment loader, and CORS-enabled FastAPI lifespan.

• **Component 2: Pydantic v2 Models & Schema Architecture:** Designed strongly-typed data schemas in `app/models/resume.py` and `app/models/job_description.py` encapsulating raw text, parsing metadata, contact info (emails, phones, URLs, location), work experience, education, and 6-domain categorized skills.

• **Component 3: Hybrid Document Parsing & OCR Pipeline:** Constructed resilient multi-format document parser in `app/services/parser.py`. Automatically routes native PDFs to pdfplumber, Word documents to python-docx, and flat/scanned image files (PNG, JPG, scanned PDFs) to Tesseract OCR via pdf2image when extracted text density is under 50 characters.

• **Component 4: NER & Skills Taxonomy Extraction Engine:** Engineered spaCy NLP pipeline (`en_core_web_sm`) in `app/services/ner.py` featuring PhraseMatcher and EntityRuler against a curated taxonomy (`skills_taxonomy.json`) containing 1000+ technical skills categorized into 6 domains: Languages, Frameworks, Databases, DevOps/Cloud, Tools/Platforms, and Methodologies. Integrated regex matchers for candidate emails, phone numbers, and profile URLs.

• **Component 5: Job Description Extraction Engine:** Created `app/services/jd_parser.py` and router `app/routers/jds.py` which analyze raw job description text to segment required vs. preferred skills, minimum/maximum experience year constraints, and degree qualifications.

• **Component 6: Asynchronous REST API Endpoints:** Implemented FastAPI routers for resume upload, retrieval, pagination, and deletion (`/api/resumes`), job descriptions management (`/api/jds`), and system liveness verification (`/health`).

• **Component 7: Dataset & Sample Synthesis Scripts:** Authored `scripts/create_samples.py` to programmatically generate realistic multi-format resumes across PDF, DOCX, and scanned image formats. Provided `scripts/download_dataset.py` for downloading large-scale public resume datasets.

• **Component 8: Modern React Workstation Frontend:** Built responsive Single Page Application using React 19 and Vite 8 following the custom warm terracotta design system. Key UI capabilities: drag-and-drop file upload with format pills, real-time KPI metrics bar (resumes count, skills count, avg density), search with instant keyword filtering, format tabs, ranked candidate dossiers with match index dials, detailed NER & raw text inspector modal, and functional CSV export.

• **Component 9: Test Suite & Build Verification:** Created comprehensive pytest suite (`backend/tests`) testing API health checks, multi-format parser edge cases, and NER skills categorization. Verified zero-error production build (`npm run build` in 244ms).

3. System Architecture & Technology Matrix

Subsystem Layer	Technology Stack	Phase 1 Role	Status
Backend API	Python 3.11 + FastAPI + Uvicorn	Asynchronous REST routing, lifespan, file streaming	COMPLETED
Database Layer	MongoDB 7.0 + Motor Async Driver	NoSQL persistence for parsed resumes & JDs	COMPLETED
Native Document Parsing	pdfplumber + python-docx	Digital layout decoding & clean text extraction	COMPLETED
Optical Character Rec.	pytesseract + pdf2image + Pillow	Automated fallback OCR for scanned documents	COMPLETED
NLP & Skills NER	spaCy (en_core_web_sm) + PhraseMatcher	Entity extraction + 1000+ skills taxonomy	COMPLETED
Frontend Workstation	React 19 + Vite 8 + Tailwind CSS	Terracotta themed recruiter workstation & dossiers	COMPLETED
Semantic Embeddings	Sentence-BERT (all-MiniLM-L6-v2)	Dense vector embedding generation	PHASE 2
Similarity & Ranking	Cosine Similarity + Weighted Scoring	Candidate ranking against Job Requirements	PHASE 2
Fairness & De-biasing	PII Redactor + Demographic Parity Audit	Equal opportunity & demographic bias mitigation	PHASE 3

4. Remaining Work & Development Roadmap

With the solid foundational architecture established in Phase 1, the remaining work is organized into three sequential phases representing Weeks 5 through 12 of the academic project lifecycle:

Phase 2 (Weeks 5–7): Semantic Embeddings & Candidate Ranking Algorithm

• **Sentence-BERT Integration:** Load `sentence-transformers` (`all-MiniLM-L6-v2`) to compute 384-dimensional dense semantic vectors for both candidate profiles and job descriptions.

• **Hybrid Similarity Model:** Implement cosine similarity matching combined with rule-based weighted scoring: *Score = w1*(Semantic Sim) + w2*(Must-Have Skills %) + w3*(Experience Fit %)*.

• **Candidate Scoring & Ranking Service:** Expose `/api/rankings` endpoint returning ranked candidate lists, score breakdowns, and missing skill gap analysis.

• **Vector Caching:** Cache precomputed resume embeddings in MongoDB/NumPy arrays to ensure sub-100ms ranking latency.

Phase 3 (Weeks 8–10): Algorithmic Fairness, Bias Auditing & Explainable AI

• **PII & Demographic Redaction:** Complete `app/services/redactor.py` to anonymize gender, ethnicity proxies, candidate names, addresses, and graduation dates prior to ranking.

• **Demographic Parity Audit:** Implement fairness evaluation metrics (Disparate Impact Ratio, Four-Fifths Rule compliance) across simulated demographic subgroups.

• **Explainable AI (XAI) Badges:** Provide transparent explanations for why each candidate received their ranking (e.g. key contributing skills, experience qualification factors).

Phase 4 (Weeks 11–12): Recruiter Analytics Dashboard & Final Capstone Deployment

• **Recruiter Comparison Matrix:** Interactive side-by-side comparison modal allowing recruiters to evaluate multiple short-listed candidates against the same job description.

• **Full Docker Compose Stack:** Containerize frontend with Nginx, backend with Uvicorn, and MongoDB with persistent data volumes.

• **Documentation & Final Presentation:** Complete college project report, IEEE-format research paper draft, presentation slides, and live demonstration sandbox.

5. Overall Project Schedule & Deliverables

Phase / Milestone	Weeks	Target Key Deliverables	Status
Phase 1: Ingestion, OCR & NER	Weeks 1–4	Parser, OCR, spaCy NER, MongoDB, React Workstation UI	**100% COMPLETE**
Phase 2: Embeddings & Ranking	Weeks 5–7	Sentence-BERT, Cosine Matching, Weighted Scoring Engine	**PLANNED NEXT**
Phase 3: Bias Mitigation & XAI	Weeks 8–10	PII Redaction, Demographic Parity, Explainable AI Badges	**UPCOMING**
Phase 4: Dashboard & Final Release	Weeks 11–12	Side-by-side Recruiter UI, Docker deployment, Project Report	**UPCOMING**

Summary & Next Steps: Phase 1 has established an end-to-end operational baseline. The codebase is cleanly decoupled and prepared for Phase 2 Sentence-BERT vectorization and ranking algorithms.